

SQL Window Functions Intro

ROW_NUMBER | RANK | LAG

Analytical SQL patterns for business questions, applied on an AdventureWorks-style sales dataset.

PREPARED BY

Akshat Srivastava

TRACK

Data Analytics

TASK

Task 23 - Level 2

DATE

30 September 2026

Overview

Objective, dataset and approach

Objective

Learn analytical SQL patterns by using **window functions** (ROW_NUMBER, RANK and LAG) to answer real business questions such as top products, best customers, growth trends and customer behaviour.

\$18,967,997 Total revenue	4,480 Orders	500 Customers	12 Queries written
--------------------------------------	------------------------	-------------------------	------------------------------

Dataset

Item	Details
Source	AdventureWorks-style sales data (synthetic, simplified)
Size	11,124 order lines 4,480 orders Jan 2024 to Sep 2026
Entities	500 customers 38 products 4 categories 10 territories 10 salespeople
Environment	Google Colab with Python (pandas) and SQLite
Main table	sales (all 8 source tables joined into one flat table)

Approach

- Loaded the CSV into a pandas DataFrame and pushed it into an in-memory SQLite table named **sales**.
- Wrote 12 SQL queries covering ranking, trend and cumulative analysis.
- Verified each output and recorded the business insight it provides.

Functions Learned

Quick reference of every window function used

Function	What it does	Business use
ROW_NUMBER()	Gives every row a unique number	Top N per group, first order per customer
RANK()	Same rank for ties, next rank is skipped (1, 1, 3)	Ranking products, customers, salespeople
DENSE_RANK()	Same rank for ties, no rank skipped (1, 1, 2)	Ranking without gaps
LAG()	Reads the previous row's value	MoM / YoY growth, gap between orders
SUM() OVER()	Running (cumulative) total	Cumulative revenue
AVG() OVER()	Moving average over a window frame	Smoothing monthly trends
PARTITION BY	Restarts the calculation for each group	Per customer, category or territory

Why window functions? Unlike GROUP BY, they calculate across related rows **without collapsing them**. That makes rankings, trends and running totals simpler and cleaner than self-joins or subqueries.

Ranking functions on tied data (Q4)

Four customers each placed the same number of orders. Notice how the three functions treat the tie differently:

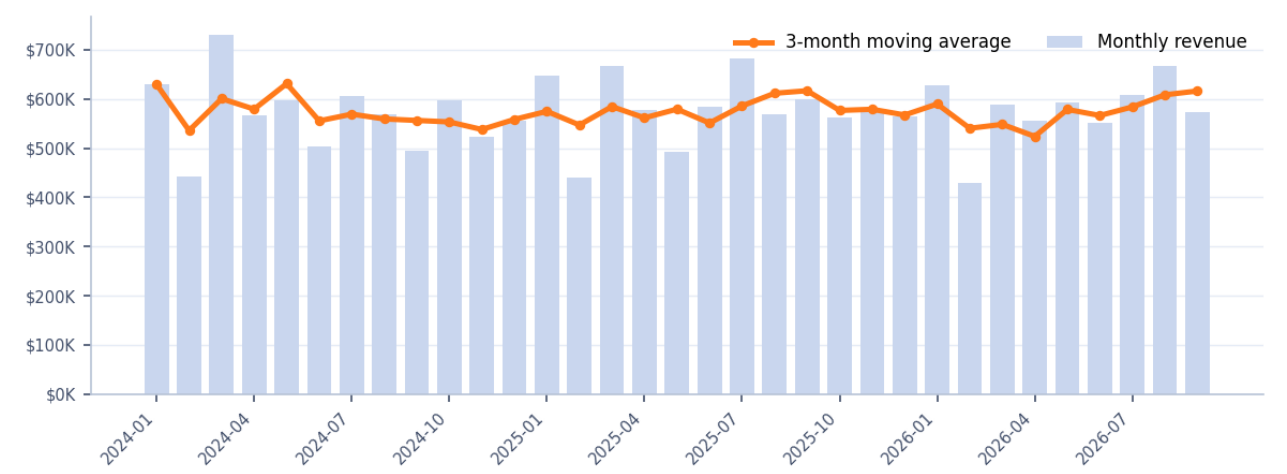
CustomerName	total_orders	row_num	rnk	dense_rnk
Amy Mitchell	20	1	1	1
Shu Mensa-Annan	20	2	1	1
Tsvi Valdez	18	3	3	2
Carol Campbell	18	4	3	2

Key Insights

What the data tells us

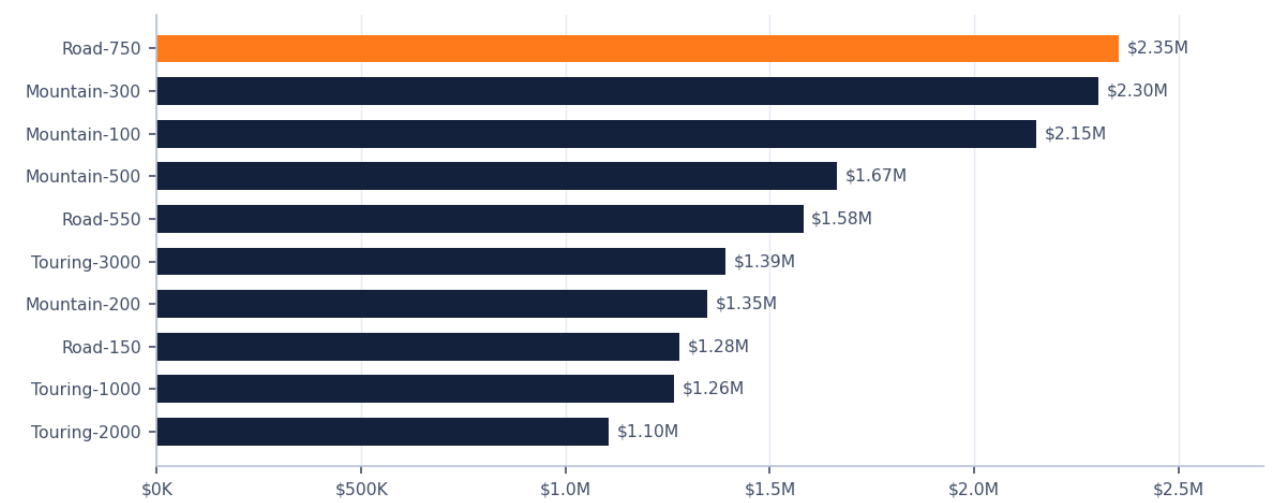
Best product	Road-750 led revenue with \$2.35M, followed closely by Mountain-300 at \$2.30M.
Top salesperson	Jose Carson ranked first with \$2.12M in sales, ahead of Tete Ansman-Wolfe (\$2.03M).
Monthly swings	Revenue is volatile: 2025-02 fell 32.0% while 2024-03 rose 65.0% month over month. The moving average reveals the underlying trend.
Ranking ties	Several customers share 20 orders. ROW_NUMBER separated them arbitrarily, while RANK and DENSE_RANK gave them the same rank.

Monthly revenue with 3-month moving average (Q7, Q11)



Bars show raw monthly revenue; the orange line smooths short-term noise.

Top 10 products by revenue (Q3)

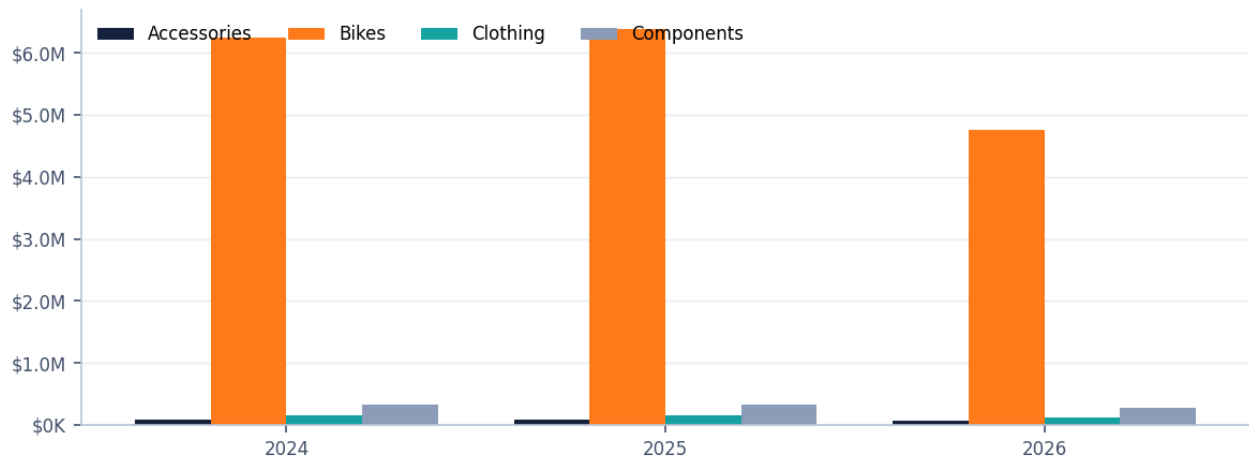


Highlighted bar marks the #1 product.

Trends and Performance

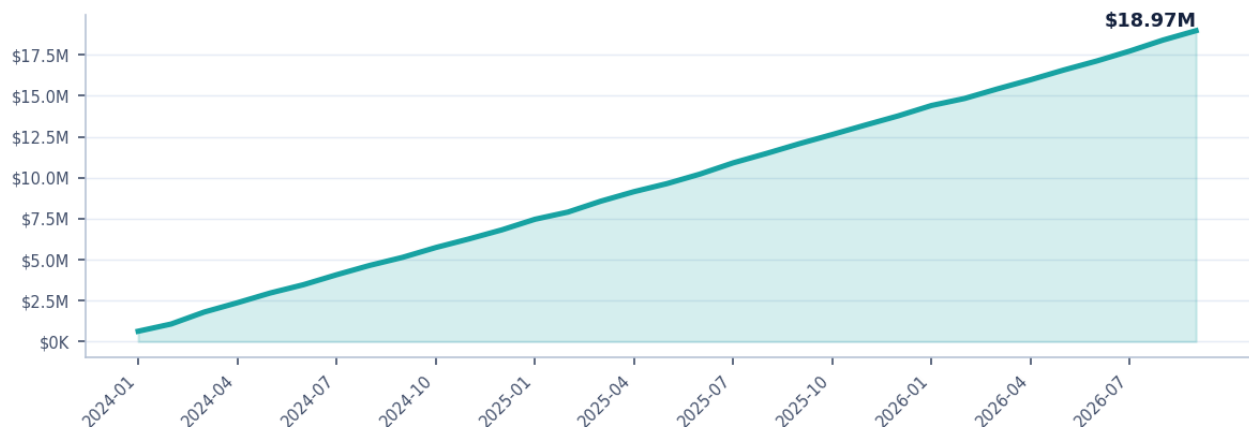
Visual summary of growth and rankings

Yearly revenue by category (Q9)



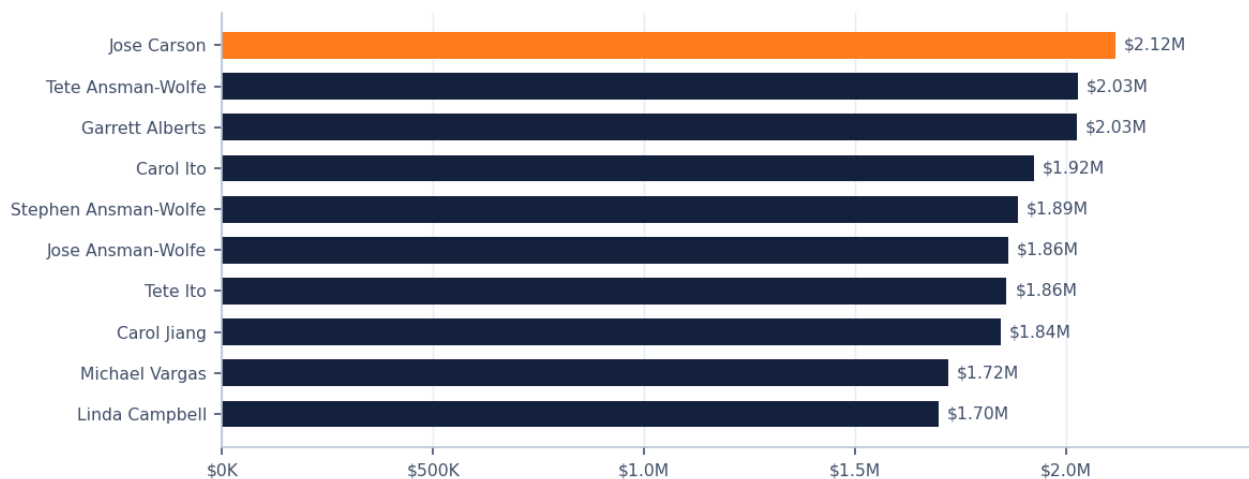
Compare each category against the previous year to see YoY movement. 2026 covers Jan to Sep only.

Cumulative revenue (Q10)



Running total keeps rising steadily across the full period.

Sales by salesperson (Q6)



Highlighted bar marks the top performer.

Query Walkthrough

All 12 queries with their outputs

Q1 Order number per customer

ROW_NUMBER()

Numbers each customer's orders 1, 2, 3 in date order. PARTITION BY restarts the count for every customer.

```
SELECT CustomerName, SalesOrderID, OrderDate, TotalDue,
       ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate, SalesOrderID) AS order_no
FROM (SELECT DISTINCT SalesOrderID, OrderDate, CustomerID, CustomerName, TotalDue FROM sales)
ORDER BY CustomerID, order_no
LIMIT 20
```

Output (first 6 of 20 rows)

CustomerName	SalesOrderID	OrderDate	TotalDue	order_no
Syed Jiang	11	2024-01-03	321.20	1
Syed Jiang	186	2024-02-12	146.33	2
Syed Jiang	203	2024-02-15	842.13	3
Syed Jiang	574	2024-05-10	5,687.58	4
Syed Jiang	773	2024-06-25	16,517.11	5
Syed Jiang	1037	2024-08-22	8,135.48	6

Q2 Top 3 products in each category

ROW_NUMBER()

Finds the three best-selling products inside every category by revenue.

```
SELECT * FROM (
  SELECT Category, ProductName, ROUND(SUM(LineTotal),2) AS revenue,
         ROW_NUMBER() OVER (PARTITION BY Category ORDER BY SUM(LineTotal) DESC) AS rn
  FROM sales GROUP BY Category, ProductName
) WHERE rn <= 3
ORDER BY Category, rn
```

Output (first 6 of 12 rows)

Category	ProductName	revenue	rn
Accessories	Pro Helmet	65,941.34	1
Accessories	Sport-200 Helmet	45,005.47	2
Accessories	Sport-100 Helmet	38,842.56	3
Bikes	Road-750	2,353,371.79	1
Bikes	Mountain-300	2,302,925.31	2
Bikes	Mountain-100	2,152,443.09	3

Q3 Product revenue ranking**RANK()**

Ranks all products by total revenue. Equal revenue gets the same rank and the next rank is skipped.

```
SELECT ProductName, Category, ROUND(SUM(LineTotal),2) AS revenue,
       RANK() OVER (ORDER BY SUM(LineTotal) DESC) AS revenue_rank
FROM sales GROUP BY ProductName, Category
ORDER BY revenue_rank
LIMIT 15
```

Output (first 6 of 15 rows)

ProductName	Category	revenue	revenue_rank
Road-750	Bikes	2,353,371.79	1
Mountain-300	Bikes	2,302,925.31	2
Mountain-100	Bikes	2,152,443.09	3
Mountain-500	Bikes	1,665,157.28	4
Road-550	Bikes	1,581,650.72	5
Touring-3000	Bikes	1,390,368.36	6

Q4 ROW_NUMBER vs RANK vs DENSE_RANK**3 ranking functions**

Compares the three ranking functions on customers with tied order counts. ROW_NUMBER is always unique, RANK leaves gaps after ties, DENSE_RANK leaves no gaps.

```
SELECT CustomerName, COUNT(DISTINCT SalesOrderID) AS total_orders,
       ROW_NUMBER() OVER (ORDER BY COUNT(DISTINCT SalesOrderID) DESC) AS row_num,
       RANK() OVER (ORDER BY COUNT(DISTINCT SalesOrderID) DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY COUNT(DISTINCT SalesOrderID) DESC) AS dense_rnk
FROM sales GROUP BY CustomerID, CustomerName
ORDER BY row_num
LIMIT 15
```

Output (first 8 of 15 rows)

CustomerName	total_orders	row_num	rnk	dense_rnk
Amy Mitchell	20	1	1	1
Shu Mensa-Annan	20	2	1	1
Tsvi Valdez	18	3	3	2
Carol Campbell	18	4	3	2
Syed Blythe	18	5	3	2
Linda Campbell	17	6	6	3
Jose Varkey Chudukatil	17	7	6	3
Syed Jiang	16	8	8	4

Q5 Top 3 customers per territory**RANK() + PARTITION BY**

Ranks customers by revenue separately inside each sales territory and keeps the top three.

```
SELECT * FROM (
  SELECT TerritoryName, CustomerName, ROUND(SUM(LineTotal),2) AS revenue,
         RANK() OVER (PARTITION BY TerritoryName ORDER BY SUM(LineTotal) DESC) AS rnk
  FROM sales GROUP BY TerritoryName, CustomerID, CustomerName
) WHERE rnk <= 3
ORDER BY TerritoryName, rnk
```

Output (first 6 of 30 rows)

TerritoryName	CustomerName	revenue	rnk
Australia	Shu Campbell	106,418.27	1
Australia	Lynn Ito	89,326.65	2
Australia	Amy Mitchell	81,546.08	3
Canada	Syed Saraiva	92,937.27	1
Canada	Syed Blythe	86,748.72	2
Canada	Linda Varkey Chudukatil	85,833.47	3

Q6 Salesperson ranking vs quota**RANK()**

Ranks salespeople by total sales and shows the share of quota achieved.

```
SELECT SalesPersonName, SalesQuota, ROUND(SUM(LineTotal),2) AS total_sales,
       ROUND(SUM(LineTotal)*100.0/SalesQuota,1) AS pct_of_quota,
       RANK() OVER (ORDER BY SUM(LineTotal) DESC) AS sales_rank
FROM sales GROUP BY SalesPersonID, SalesPersonName, SalesQuota
ORDER BY sales_rank
```

Output (first 6 of 10 rows)

SalesPersonName	SalesQuota	total_sales	pct_of_quota	sales_rank
Jose Carson	750,000.00	2,116,295.67	282.20	1
Tete Ansman-Wolfe	750,000.00	2,028,981.35	270.50	2
Garrett Alberts	500,000.00	2,026,061.42	405.20	3
Carol Ito	500,000.00	1,924,046.19	384.80	4
Stephen Ansman-Wolfe	650,000.00	1,886,603.74	290.20	5
Jose Ansman-Wolfe	500,000.00	1,862,721.91	372.50	6

Q7 Month-over-month growth

LAG()

Brings the previous month's revenue into the current row to calculate growth %. The first row is NULL because no earlier month exists.

```
WITH monthly AS (  
    SELECT SUBSTR(OrderDate,1,7) AS month, ROUND(SUM(LineTotal),2) AS revenue  
    FROM sales GROUP BY 1  
)  
SELECT month, revenue,  
    LAG(revenue) OVER (ORDER BY month) AS prev_month_revenue,  
    ROUND((revenue - LAG(revenue) OVER (ORDER BY month)) * 100.0  
        / LAG(revenue) OVER (ORDER BY month), 2) AS mom_growth_pct  
FROM monthly  
ORDER BY month
```

Output (first 6 of 33 rows)

month	revenue	prev_month_revenue	mom_growth_pct
2024-01	629,742.11	NULL	NULL
2024-02	442,282.02	629,742.11	-29.77
2024-03	729,931.29	442,282.02	65.04
2024-04	565,972.42	729,931.29	-22.46
2024-05	597,810.96	565,972.42	5.63
2024-06	503,120.69	597,810.96	-15.84

Q8 Gap between customer orders

LAG() + PARTITION BY

Fetches each customer's previous order date and calculates the number of days between two orders.

```
SELECT CustomerName, OrderDate,  
    LAG(OrderDate) OVER (PARTITION BY CustomerID ORDER BY OrderDate, SalesOrderID) AS prev_order_date,  
    CAST(julianday(OrderDate) - julianday(  
        LAG(OrderDate) OVER (PARTITION BY CustomerID ORDER BY OrderDate, SalesOrderID)) AS INT) AS days_gap  
FROM (SELECT DISTINCT SalesOrderID, OrderDate, CustomerID, CustomerName FROM sales)  
ORDER BY CustomerID, OrderDate  
LIMIT 20
```

Output (first 6 of 20 rows)

CustomerName	OrderDate	prev_order_date	days_gap
Syed Jiang	2024-01-03	NULL	NULL
Syed Jiang	2024-02-12	2024-01-03	40.00
Syed Jiang	2024-02-15	2024-02-12	3.00
Syed Jiang	2024-05-10	2024-02-15	85.00
Syed Jiang	2024-06-25	2024-05-10	46.00
Syed Jiang	2024-08-22	2024-06-25	58.00

Q9 Category-wise year-over-year growth**LAG() + PARTITION BY**

Compares each category's yearly revenue with the previous year to get YoY growth %.

```
WITH yearly AS (
  SELECT Category, SUBSTR(OrderDate,1,4) AS year, ROUND(SUM(LineTotal),2) AS revenue
  FROM sales GROUP BY Category, year
)
SELECT Category, year, revenue,
  LAG(revenue) OVER (PARTITION BY Category ORDER BY year) AS prev_year_revenue,
  ROUND((revenue - LAG(revenue) OVER (PARTITION BY Category ORDER BY year)) * 100.0
    / LAG(revenue) OVER (PARTITION BY Category ORDER BY year), 2) AS yoy_growth_pct
FROM yearly
ORDER BY Category, year
```

Output (first 8 of 12 rows)

Category	year	revenue	prev_year_revenue	yoy_growth_pct
Accessories	2024	80,300.78	NULL	NULL
Accessories	2025	85,788.86	80,300.78	6.83
Accessories	2026	60,218.83	85,788.86	-29.81
Bikes	2024	6,245,384.07	NULL	NULL
Bikes	2025	6,383,083.67	6,245,384.07	2.20
Bikes	2026	4,750,509.89	6,383,083.67	-25.58
Clothing	2024	154,670.33	NULL	NULL
Clothing	2025	161,216.20	154,670.33	4.23

Q10 Running total of revenue**SUM() OVER()**

Adds up revenue month by month to show cumulative growth.

```
WITH monthly AS (
  SELECT SUBSTR(OrderDate,1,7) AS month, SUM(LineTotal) AS revenue FROM sales GROUP BY 1
)
SELECT month, ROUND(revenue,2) AS revenue,
  ROUND(SUM(revenue) OVER (ORDER BY month),2) AS running_total
FROM monthly
ORDER BY month
```

Output (first 6 of 33 rows)

month	revenue	running_total
2024-01	629,742.11	629,742.11
2024-02	442,282.02	1,072,024.13
2024-03	729,931.29	1,801,955.42
2024-04	565,972.42	2,367,927.84
2024-05	597,810.96	2,965,738.80
2024-06	503,120.69	3,468,859.49

Q11 3-month moving average

AVG() OVER() + frame

Smooths monthly ups and downs. The frame covers the current month and the previous two months.

```
WITH monthly AS (  
    SELECT SUBSTR(OrderDate,1,7) AS month, SUM(LineTotal) AS revenue FROM sales GROUP BY 1  
)  
SELECT month, ROUND(revenue,2) AS revenue,  
        ROUND(AVG(revenue) OVER (ORDER BY month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW),2) AS moving_avg_3m  
FROM monthly  
ORDER BY month
```

Output (first 6 of 33 rows)

month	revenue	moving_avg_3m
2024-01	629,742.11	629,742.11
2024-02	442,282.02	536,012.06
2024-03	729,931.29	600,651.81
2024-04	565,972.42	579,395.24
2024-05	597,810.96	631,238.22
2024-06	503,120.69	555,634.69

Q12 First order of every customer

ROW_NUMBER() = 1

Picks only the first order per customer, the standard trick for getting the first row of each group.

```
SELECT * FROM (  
    SELECT CustomerName, SalesOrderID, OrderDate, TotalDue,  
           ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate, SalesOrderID) AS rn  
    FROM (SELECT DISTINCT SalesOrderID, OrderDate, CustomerID, CustomerName, TotalDue FROM sales)  
) WHERE rn = 1  
ORDER BY OrderDate  
LIMIT 20
```

Output (first 6 of 20 rows)

CustomerName	SalesOrderID	OrderDate	TotalDue	rn
Tsvi Carson	2	2024-01-01	5,909.60	1
Lynn Saraiva	3	2024-01-01	298.43	1
Rachel Tsolfias	1	2024-01-01	4,497.95	1
Amy Saraiva	6	2024-01-02	302.00	1
Ranjit Ansman-Wolfe	5	2024-01-02	2,870.36	1
Jose Campbell	4	2024-01-02	9,533.92	1

Conclusion

Summary and final takeaway

What I achieved

- Applied **ROW_NUMBER**, **RANK** and **DENSE_RANK** to rank products, customers and salespeople, including per-group top N results.
- Used **LAG** to compute month-over-month growth, year-over-year growth and days between customer orders.
- Built **running totals** and a **3-month moving average** using window frames.
- Learned to separate order-level and line-level data to avoid double counting.

Challenges and notes

- The flat table repeats order-level values (SubTotal, TotalDue) on every line of an order. I used **SELECT DISTINCT** for order-level analysis and **LineTotal** for revenue.
- LAG returns NULL for the first row of each group because no earlier row exists.
- The dataset is synthetic, so quota percentages are illustrative and not real business results.

Final takeaway

Window functions let me calculate rankings, trends and running totals without collapsing rows like GROUP BY does. They are cleaner than self-joins and subqueries, and essential for real analytical work such as finding top performers, tracking growth and studying customer behaviour.